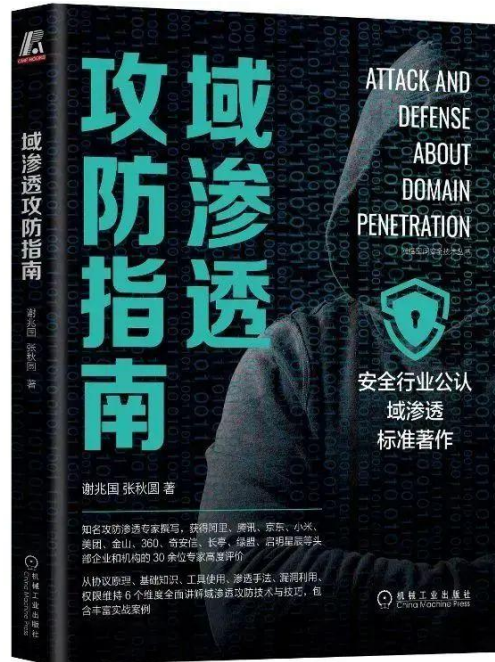


证书服务基础知识

本文部分节选于《域渗透攻防指南》，购买请长按如下图片扫码



小谢1997521 为你推荐

【限量签名版】《域渗透攻防指南》

¥99 ¥129



本商品由 放心购 提供保障

谢公子学安全

2021年6月17日，国外安全研究员 Will Schroeder 和 Lee Christensen 共同发布了针对 Active Directory 证书服务(ADCS: Active Directory Certificate Service)的攻击手法：

https://www.specterops.io/assets/resources/Certified_Pre-Owned.pdf。并

且 Will Schroeder 和 Lee Christensen 于 8 月 5 日在 BlackHat2021 上对该攻击手法进行了详细讲解：[https://www.blackhat.com/us-](https://www.blackhat.com/us-21/briefings/schedule/#certified-pre-owned-abusing-active-directory-)

[21/briefings/schedule/#certified-pre-owned-abusing-active-directory-](https://www.blackhat.com/us-21/briefings/schedule/#certified-pre-owned-abusing-active-directory-)

[certificate-services-23168](#)。至此，ADCS 证书服务攻击第一次进入人们的视野。

PKI 公钥基础设施

PKI (Public Key Infrastructure) 公钥基础设施，是提供公钥加密和数字签名服务的系统或平台，是一个包括硬件、软件、人员、策略和规程的集合，用来实现基于公钥密码体制的密钥和证书的产生、管理、存储、分发和撤销等功能。企业通过采用 PKI 框架管理密钥和证书可以建立一个安全的网络环境。

PKI 的基础技术包括：公钥加密、数字签名、数据完整性机制、数字信封(混合加密)、双重数字签名等。

PKI 体系能够实现的功能有：

- 身份验证；
- 数据完整性；
- 数据机密性；
- 操作的不可否认性。

微软的活动目录证书服务 ADCS 就是对 PKI 的实现，活动目录证书服务能够跟现有的活动目录域服务(ADDS: Active Directory Domain Service)进行结合，可以用于身份验证、公钥加密和数字签名等。ADCS 提供所有与 PKI 相关的组件作为角色服务。每个角色服务负责证书基础架构的特定部分，同时协同工作以形成完整的解决方案。

ADCS 允许你做如下的操作：

- 设置 Web 注册、网络设备注册服务和在线响应器服务。
- 管理用户、计算机、服务和路由器等网络设备的证书的注册和吊销。
- 使用组策略来分发和管理证书。

CA 证书颁发机构

CA (Certificate Authority, 证书颁发机构) 是 PKI 系统的核心。其作用包括处理证书申请、证书发放、证书更新、管理已颁发的证书、吊销证书和发布证书吊销列表(CRL)等。

Active Directory 证书服务中的 CA 有企业 CA 和独立 CA。企业 CA 必须是域成员, 并且通常处于联机状态以颁发证书或证书策略。而独立 CA 可以是成员、工作组或域。独立 CA 不需要 ADDS 活动目录域服务, 并且可以在没有网络的情况下使用。但是在域中基本都是使用企业 CA, 因为企业 CA 可以和活动目录域服务 ADDS 进行结合, 其信息也存储在 Active Directory 数据库中。企业 CA 支持基于证书模块创建证书和自动注册证书。

如图所示, 可以看到在安装活动目录证书服务时可供选择的企业 CA(E)和独立 CA(A)。

The screenshot shows a window titled "Active Directory Certificate Services Installation Wizard". It contains two radio button options:

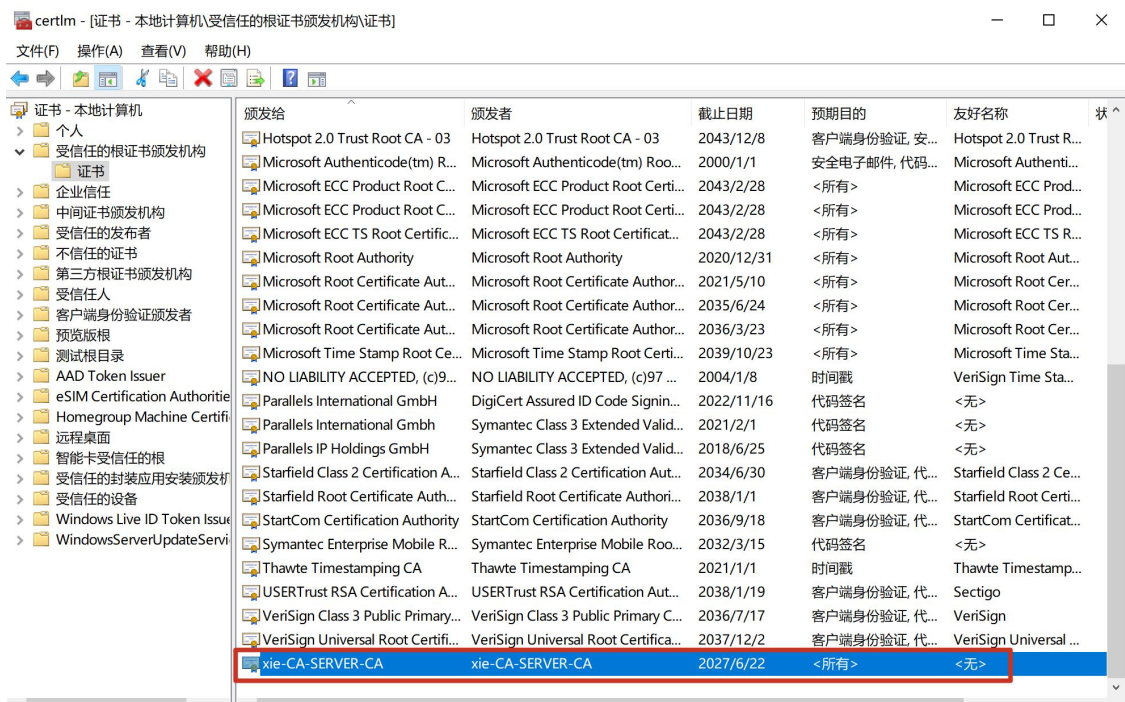
- ☒ 企业 CA(E)
企业 CA 必须是域成员, 并且通常处于联机状态以颁发证书或证书策略。
- ☐ 独立 CA(A)
独立 CA 可以是成员、工作组或域。独立 CA 不需要 AD DS, 可在没有网络连接的情况(脱机)下使用。

在创建 CA 后, 会自动创建 CA 的公私钥对:

- 私钥只有 CA 知道, 私钥用于对颁发的证书进行数字签名。
- 公钥任何人都可以知道, 公钥用于验证证书是否由 CA 颁发。

然后 CA 使用自己的私钥签署新证书来生成自己的根 CA 证书, 也就是说根 CA 证书是自签名的。ADCS 会将根 CA 证书的 Subject 和 Issuer 字段设置为 CA 的名称, 将 Basic Constraints 设置为 Subject Type=CA, 并将 NotBefore/NotAfter 字段设置为五年(默认情况下)。域内所有主机会将根 CA

证书添加到受信任的根证书颁发机构中，如下图所示。此后，该 CA 签名发行的所有证书都会被域内主机所信任。

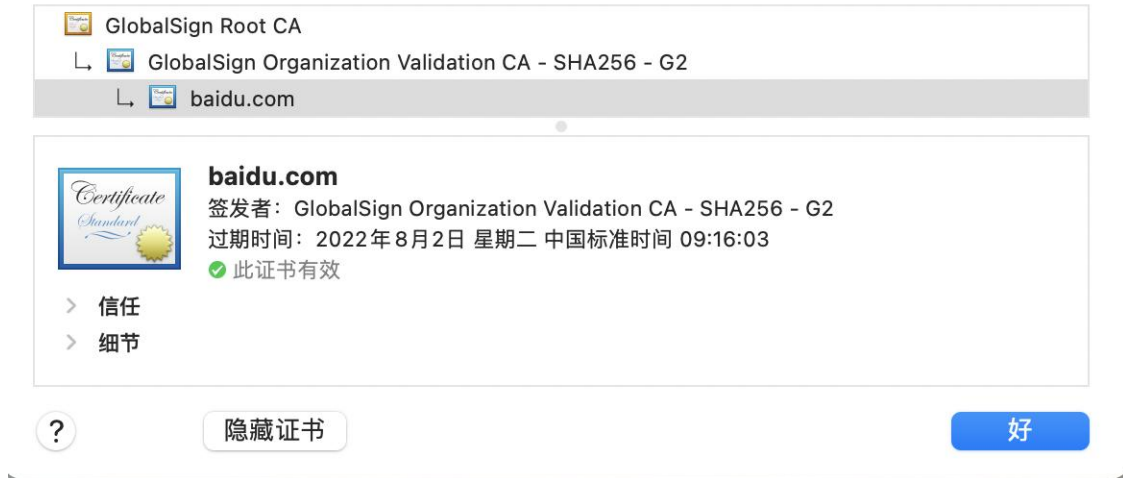


我们平时访问 https 类型的网站，如百度，通过查看其证书可以看到他的根 CA，并且显示此证书有效。如图所示：



Safari 浏览器正在使用 **www.baidu.com** 的加密连接。

使用数字证书进行加密后，在将信息发送到 https 网站“www.baidu.com”或从该网站发出信息时，信息是保密的。



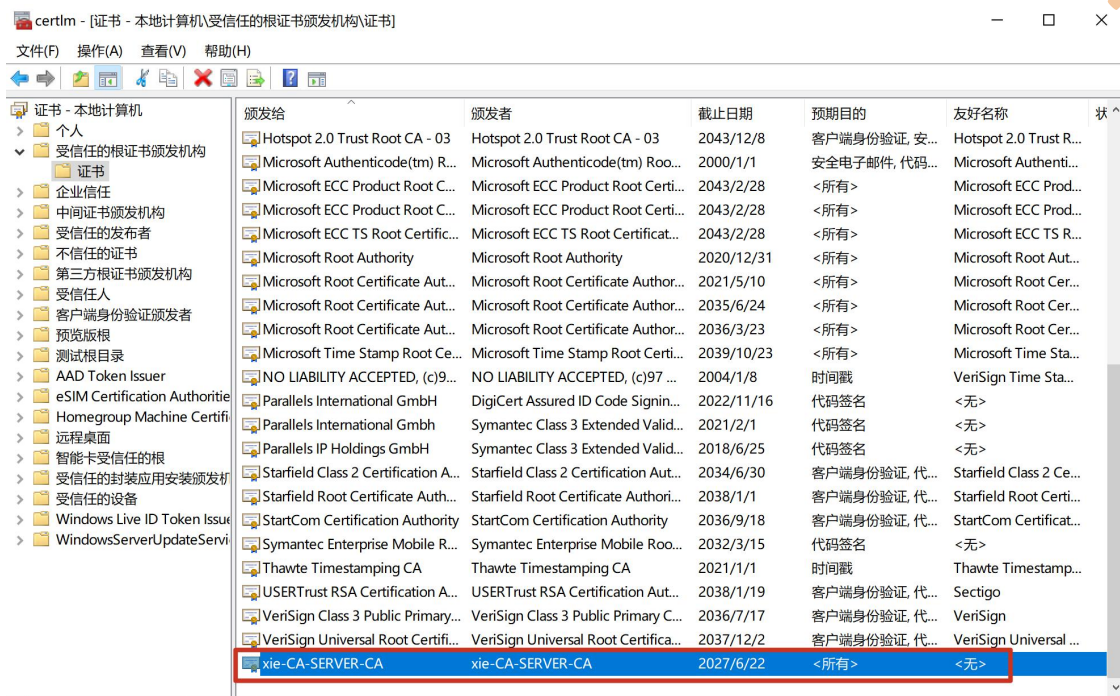
这是因为该根 CA 已经导入了我们系统信任的根证书颁发机构。因此，通过该根证书颁发机构所颁发的所有证书都是可信的。

如图所示，可以看到我们系统信任的根证书颁发机构，百度的根证书颁发机构 GlobalSign Root CA 就在其中。

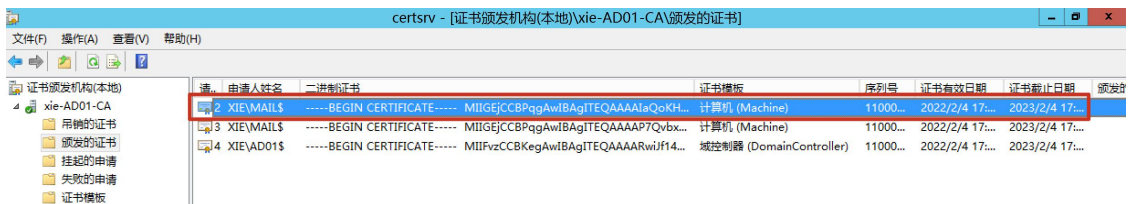
 GlobalSign Root CA 根证书颁发机构 过期时间: 2028年1月28日 星期五 中国标准时间 20:00:00 此证书有效				
名称	种类	过期时间	密钥串	
GlobalSign Root CA	证书	2028年1月28日 20:00:00	系统根证书	
GeoTrust Global CA	证书	2022年5月21日 12:00:00	系统根证书	
GeoTrust Primary Certification Authority	证书	2036年7月17日 07:59:59	系统根证书	
GeoTrust Primary Certification Authority - G2	证书	2038年1月19日 07:59:59	系统根证书	
GeoTrust Primary Certification Authority - G3	证书	2037年12月2日 07:59:59	系统根证书	
Global Chambersign Root	证书	2037年10月1日 00:14:18	系统根证书	
Global Chambersign Root - 2008	证书	2038年7月31日 20:31:40	系统根证书	
GlobalSign	证书	2038年1月19日 11:14:07	系统根证书	
GlobalSign	证书	2038年1月19日 11:14:07	系统根证书	
GlobalSign	证书	2029年3月18日 18:00:00	系统根证书	
GlobalSign	证书	2034年12月10日 08:00:00	系统根证书	
GlobalSign Root CA	证书	2028年1月28日 20:00:00	系统根证书	
Go Daddy Class 2 Certification Authority	证书	2034年6月30日 01:06:20	系统根证书	
Go Daddy Root Certificate Authority - G2	证书	2038年1月1日 07:59:59	系统根证书	
Government Root Certification Authority	证书	2037年12月31日 23:59:59	系统根证书	
GTS Root R1	证书	2036年6月22日 08:00:00	系统根证书	
GTS Root R2	证书	2036年6月22日 08:00:00	系统根证书	
GTS Root R3	证书	2036年6月22日 08:00:00	系统根证书	
GTS Root R4	证书	2036年6月22日 08:00:00	系统根证书	
Hellenic Academic and Research Institutions ECC RootCA 2015	证书	2040年6月30日 18:37:12	系统根证书	
Hellenic Academic and Research Institutions RootCA 2011	证书	2031年12月1日 21:49:52	系统根证书	
Hellenic Academic and Research Institutions RootCA 2015	证书	2040年6月30日 18:11:21	系统根证书	

而我们在活动目录内自己搭建的 ADCS 证书服务，由于在安装企业根 CA 时，系统使用组策略将根 CA 添加到域内所有机器的“受信任的根证书颁发机构”内了，因此域内机器默认都信任此根 CA 颁发的证书。

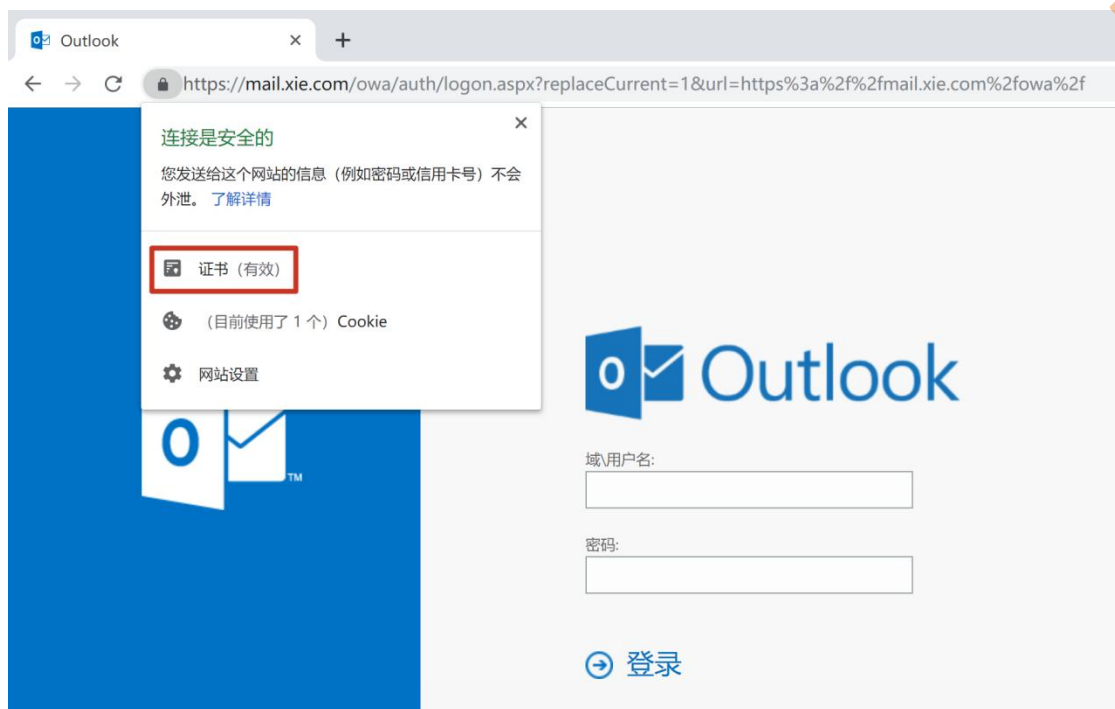
如图所示，在域内机器上的“受信任的根证书颁发机构”内可以看到我们的根 CA：xie-CA-SERVER-CA。



如图所示，在我们根 CA 颁发的证书这里可以看到已经颁发的证书，以下证书应用的网站会被域内所有机器信任。

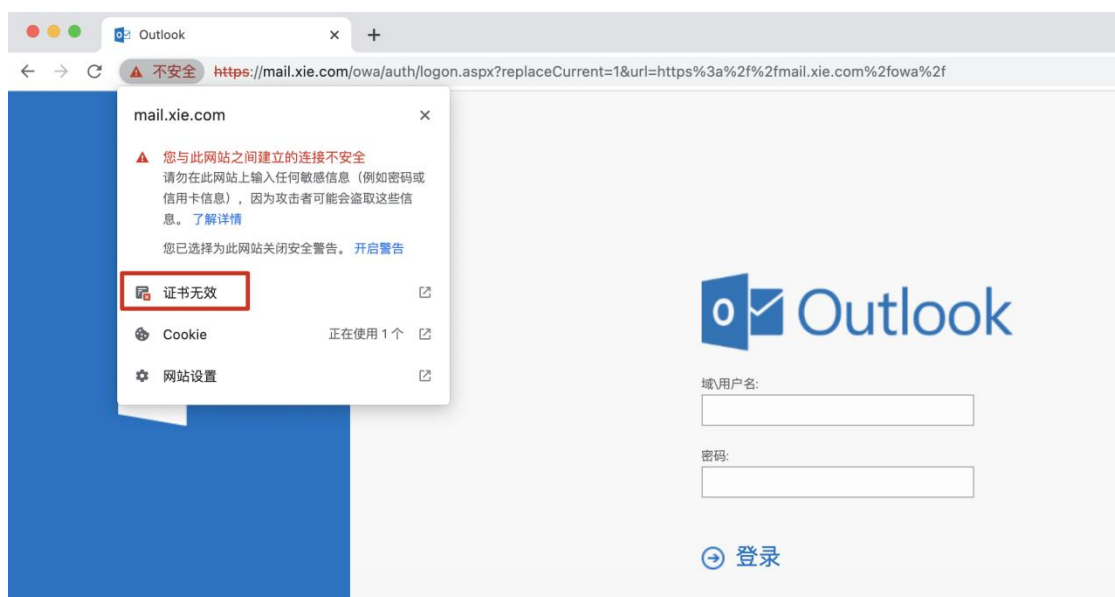


如图所示，在域内机器上访问域内的 Exchange 邮箱服务，可以看到提示证书有效。



但是，如果在域外机器访问该 Exchange 邮箱服务，会提示证书无效。

如图所示，可以看到浏览器提示证书无效。因为域外的机器并没有在“受信任的根证书颁发机构”中添加域内搭建的根 CA。



那么如何能让域外的机器也信任我们域内搭建的根 CA 颁发的证书呢？

很简单，就是把我们在域内搭建的根 CA 的证书手动导入到系统中即可。操作如下：

首先，访问 ADCS 证书服务器的/certsrv/certcarc.asp 路径，点击下载 CA 证书。如图所示：



然后将下载的 CA 证书导入系统“受信任的根证书颁发机构”内，并勾选始终信任，如图所示：

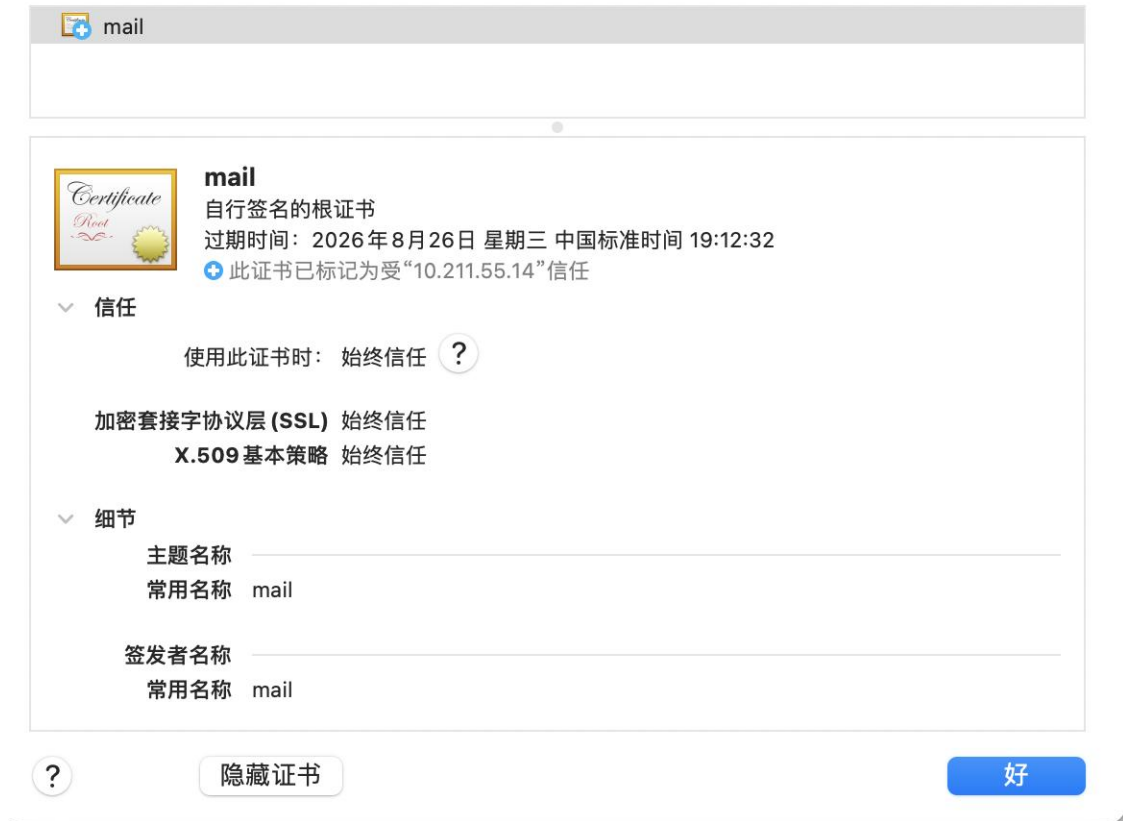


接着使用浏览器再次访问 Exchange 邮箱服务，既可以看到提示信任此证书！
如图 所示：



Safari 浏览器正在使用 mail.xie.com 的加密连接。

使用数字证书进行加密后，在将信息发送到 https 网站“mail.xie.com”或从该网站发出信息时，信息是保密的。

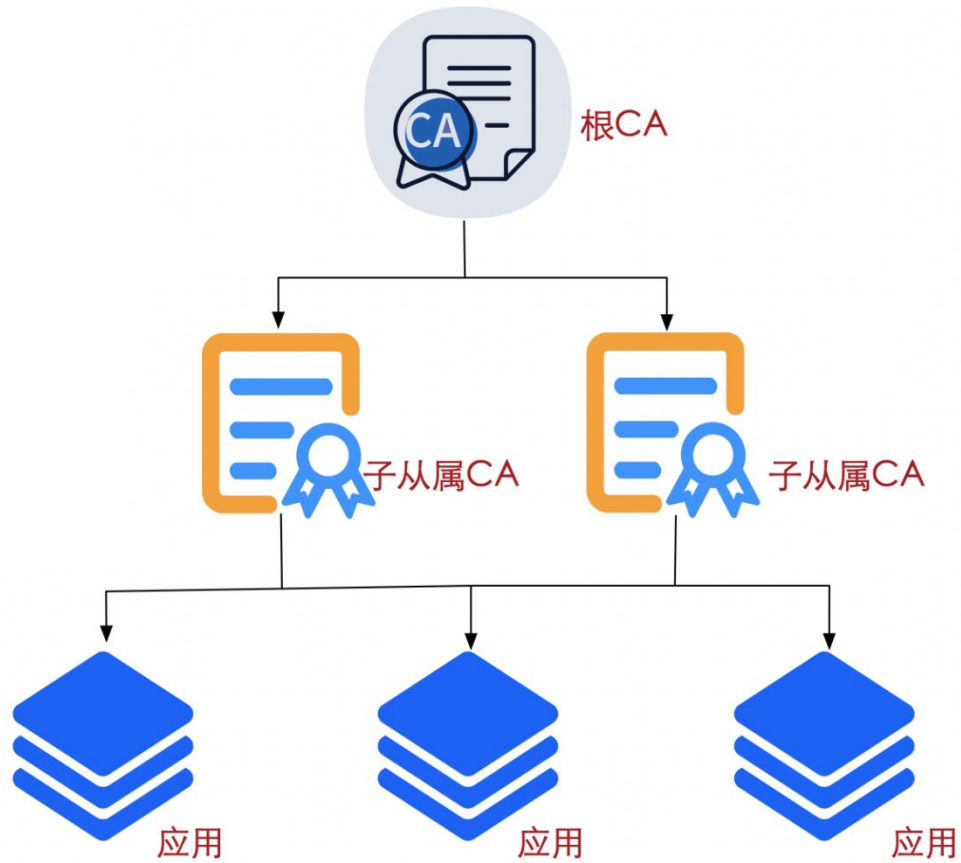


注：以上是在 Mac os 中配置受信任的根证书颁发机构，Windows 系统配置流程有差别。

CA 层次结构

常见的 CA 层次机构有两个级别，根和二级 CA，二级 CA 也叫子从属 CA。根 CA 位于顶级，子从属 CA 在第二级。在这种层次机构下，根 CA 给子从属 CA 颁发证书认证，子从属 CA 给下面的应用颁发和管理证书，根 CA 不直接给应用颁发证书。

如图所示，可以看到常见的 CA 层次结构。



CA 的层次结构，有以下优点：

- 管理层次分明，便于集中管理、政策制订和实施；
- 提高 CA 中心的总体性能、减少瓶颈；
- 有充分的灵活性和可扩展性；
- 有利于保证 CA 中心的证书验证效率；

CRL 证书作废列表

CRL (Certificate Revocation List)：证书作废列表，就是我们所称的“证书黑名单”，在证书的有效期限期间，因为某种原因（如人员调动、私钥泄漏等等），

导致相应的数字证书内容不再是真实可信。此时，进行证书撤销，说明该证书无效，CRL 中列出了被撤销的证书序列号。

证书

证书是 X.509 格式的数字签名文档，用户加密、消息签名或身份验证等功能。证书通常具有多个字段，包括如下：

- Subject：主题，可以标识证书的所有者。
- Public Key：公钥，用于将主题(Subject)与单独存储的私钥相关联。
- NotBefore and NotAfter dates：证书的有效期。
- Serial Number：CA 分配的证书标识符。
- Issuer：标识颁发证书的人(通常是 CA)。
- SubjectAlternativeName(SAN)：主题备用名称，定义一个或多个可供主题(Subject)使用的可选名称。
- Basic Constraints：基本约束，标识证书是 CA 还是最终实体，以及在使用证书时是否存在任何约束。
- Extender key Usages(EKU)：扩展密钥用法，描述证书将如何使用的对象标识符(OID)。常见的 EKU OID 如下：
 - - 代码签名 (OID 1.3.6.1.5.5.7.3.3)：证书用于签署可执行代码。
 - 加密文件系统 (OID 1.3.6.1.4.1.311.10.3.4)：证书用于加密文件系统。

— 安全电子邮件 (OID 1.3.6.1.5.5.7.3.4) : 证书用于加密电子邮件。

— 客户端身份验证 (OID 1.3.6.1.5.5.7.3.2) : 证书用于身份验证到另一个服务器。

— 智能卡登录 (OID 1.3.6.1.4.1.311.20.2.2) : 证书用于智能卡认证。

— 服务器认证 (OID 1.3.6.1.5.5.7.3.1) : 证书用于识别服务器 (例如, HTTPS 证书) 。

— PKINIT 客户端身份验证, 对应的 OID 为 1.3.6.1.5.2.3.4

— 任何目的, 对应的 OID 为 2.5.29.37.0

- Signature Algorithm: 签名算法, 指定用于签署证书的算法。
- Signature: 使用颁发者的私钥对证书进行签名。

PKINIT Kerberos 认证

在 1.2 节 Kerberos 协议篇中我们讲了在 AS-REQ 请求过程中, 是用用户 Hash 加密时间戳来进行 Kerberos 预身份认证的。而 AD CS 服务可以和 AD DS 紧密搭配使用, 那么自然会猜想, 能否利用证书来进行 Kerberos 预身份认证呢?

答案是可以的。在 RFC4556 Public Key Cryptography for Initial Authentication in Kerberos (PKINIT) 中引入了对 Kerberos 预身份验证的公钥加密技术支持, 可以使用证书的私钥来进行 Kerberos 预身份认证。

如图所示, 使用 Rubeus 执行如下命令用证书 administrator.pfx 进行 Kerberos 认证。

```
Rubeus.exe asktgt /user:Administrator /password:root /certificate:administrator.pfx /domain:xie.com /dc:DC01.xie.com
```


C:\>Rubeus.exe asktgt /user:Administrator /certificate:administrator.pfx /domain:xie.com /dc:DC01.xie.com

```

  (S)
  Rubeus
  v2.0.1

[*] Action: Ask TGT

[*] Using PKINIT with etype rc4_hmac and subject: CN=Administrator, CN=Users, DC=xie, DC=com
[*] Building AS-REQ (w/ PKINIT preauth) for: 'xie.com\Administrator'
[+] TGT request successful!
[*] base64(ticket.kirbi):

doIFpjCCBaKgAwIBBaEDAgEWooIExTCCBMFhggS9MIIeUaADAgEFoQkbB1hJRS5DT02iHDAaoAMCAQKh
EzARGwZrcmJ0Z3QbB3hpZS5jb22jggSHMIIeG6ADAgESoQMCAQKiggR1BIIecfMrf8ACgHtd/3h3TaBH
aqmPhi7a4E030f1LSJnFDjHys9Y+HDCZYyN4Ez5XuAuV/4x1Ttm1pZq4uMef79HsCyRf3C1wozkJg0Qw

```

在认证过程中使用 wireshark 抓包。如图所示，可以看到在 AS-REQ 请求包的预认证字段为 pA-PK-AS-REQ。在这个阶段，客户端发送包含证书内容的请求包，并使用证书私钥对其进行签名作为预认证数据。

```

Kerberos
  > Record Mark: 2577 bytes
  > as-req
    pvno: 5
    msg-type: krb-as-req (10)
    padata: 2 items
      > PA-DATA pA-PAC-REQUEST
      > PA-DATA pA-PK-AS-REQ
        padata-type: pA-PK-AS-REQ (16)
          padata-value: 3082094e8082094a3082094606092a864886f70d010702a082093730820933020103310b...
            > signedAuthPack (id-signedData)
            > AuthPack
    > req-body

```

KDC 在收到客户端发来的 AS-REQ 请求包后，使用证书的公钥对签名进行校验，校验通过后发送 AS-REP 回复包，其中包含 krbtgt 哈希加密的 TGT 认购权证和证书公钥加密的 Logon Session key。

并且在微软的官方文档中注意到这么一句话：为了支持连接到不支持 Kerberos 身份验证的网络服务的应用程序的 NTLM 身份验证，当使用 PKCA 时，KDC 将在 PAC 特权属性证书的 PAC_CREDENTIAL_INFO 缓冲区中返回用户的 NTLM Hash。也就是说当使用证书进行 Kerberos 认证时，返回的票据的 PAC 中 是包含用户的 NTLM Hash 的。<https://learn.microsoft.com/en->

us/openspecs/windows_protocols/ms-pkca/4e5fb325-eabc-4fac-a0da-af2b6b4430cb

1.4 Relationship to Other Protocols

Article • 05/11/2022 • 2 minutes to read

 [Feedback](#)

PKCA is defined as a Kerberos [pre-authentication](#) extension ([\[RFC4120\]](#) [↗](#) section 3.1.1). This extension is used in the Kerberos [AS exchange](#) [\[RFC4556\]](#) [↗](#), and therefore PKCA relies on a working Kerberos infrastructure and a [certificate authority \(CA\)](#) for issuing [\[X509\]](#) [↗](#) certificates. PKCA includes the use of [elliptic curve cryptography \(ECC\)](#). ECC support [\[RFC5349\]](#) [↗](#) relies upon a CA issuing ECC certificates. Applications already using Kerberos can use PKCA without modifications.

In order to support NTLM authentication [\[MS-NLMP\]](#) for applications connecting to network [services](#) that do not support Kerberos authentication, when PKCA is used, the [KDC](#) returns the user's NTLM [one-way function \(OWF\)](#) in the [privilege attribute certificate \(PAC\)](#) PAC_CREDENTIAL_INFO buffer ([\[MS-PAC\]](#) section 2.6.1).

如图所示，使用 kekeo 执行如下命令获得 administrator.pfx 证书对应的 administrator 用户的 NTLM Hash。

```
tgt::pac /subject:administrator /castore:current_user /domain:xie.com /user:administrator /cred
```

C:\>kekeo.exe

```
kekeo 2.1 (x64) built on Dec 14 2021 11:51:55
/ (')>- "A La Vie, A L'Amour"
| K | /* * *
\___/ Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
L\_ https://blog.gentilkiwi.com/kekeo (oe.eo)
      with 10 modules * * */

kekeo # tgt::pac /subject:administrator /castore:current_user /domain:xie.com /user:administrator /cred
Realm      : xie.com (xie)
User       : administrator (administrator)
CName      : administrator [KRB_NT_PRINCIPAL (1)]
SName      : krbtgt/xie.com [KRB_NT_SRV_INST (2)]
Need PAC   : Yes
Auth mode  : RSA
[kdc] name: DC01.xie.com (auto)
[kdc] addr: 10.211.55.4 (auto)
[kdc] Serial : 0f0000000000fa428b3c27c63a8c0f0000004b
[kdc] Validity: 2022/5/17 10:13:47 -> 2023/5/17 10:13:47
[kdc] Issuer : com, xie, xie-CA-SERVER-CA
[0] NTLM
NTLM: 33e17aa21ccc6ab0e6ff30eecb918dfb
```

后续无论用户密码怎么更改，使用该功能获取用户的 NTLM Hash 都是最新的

注：在使用 kekeo 获取证书对应用户的 NTLM Hash 时，需要先将证书导入到系统中。

那么，哪些模板的证书能用于 Kerberos 认证呢？

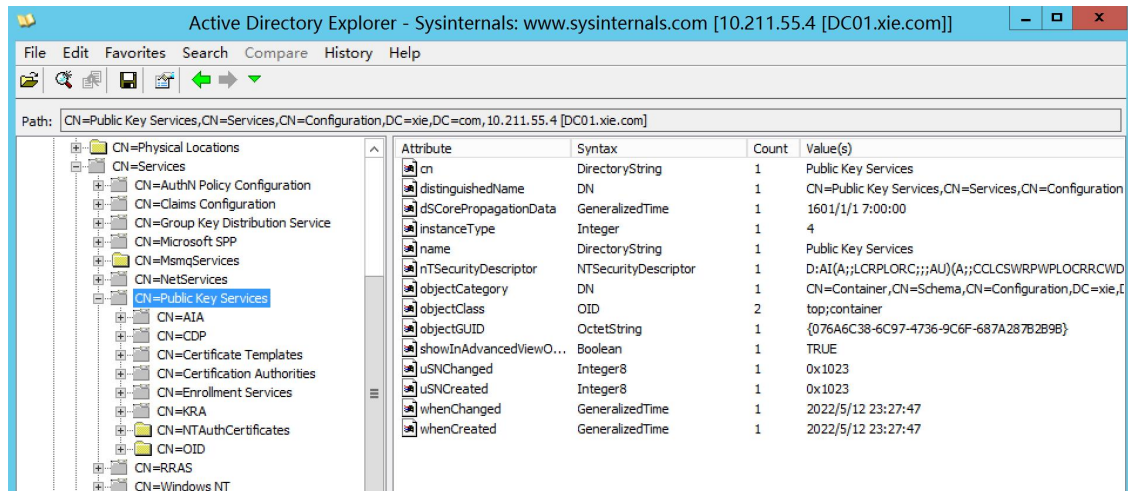
在 [Certified_Pre-Owned.pdf](#) 中提到了具有以下扩展权限的证书可以用于 Kerberos 认证：

- 客户端身份验证，对应的 OID 为 1.3.6.1.5.5.7.3.2；
- PKINIT 客户端身份验证，对应的 OID 为 1.3.6.1.5.2.3.4；
- 智能卡登录，对应的 OID 为 1.3.6.1.4.1.311.20.2.2；
- 任何目的，对应的 OID 为 2.5.29.37.0；
- 子 CA。

活动目录数据库中的 ADCS

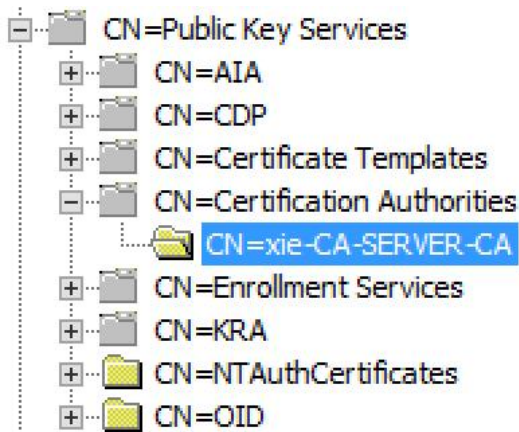
安装完 ADCS 证书服务后，可以在活动目录数据库中找到相关数据，完整路径为：CN=Public Key

Services,CN=Services,CN=Configuration,DC=xie,DC=com。如图所示：



Certification Authorities

Certification Authorities 是 CA 认证机构类，里面的实例都是受信任的 CA，AD 将这些 CA 的证书下发到到每台 Windows 计算机的受信任的根证书颁发机构证书存储区。为了使 AD 认为证书是可信的，证书的信任链最终必须以该容器中定义的根 CA 之一结束。如图所示：



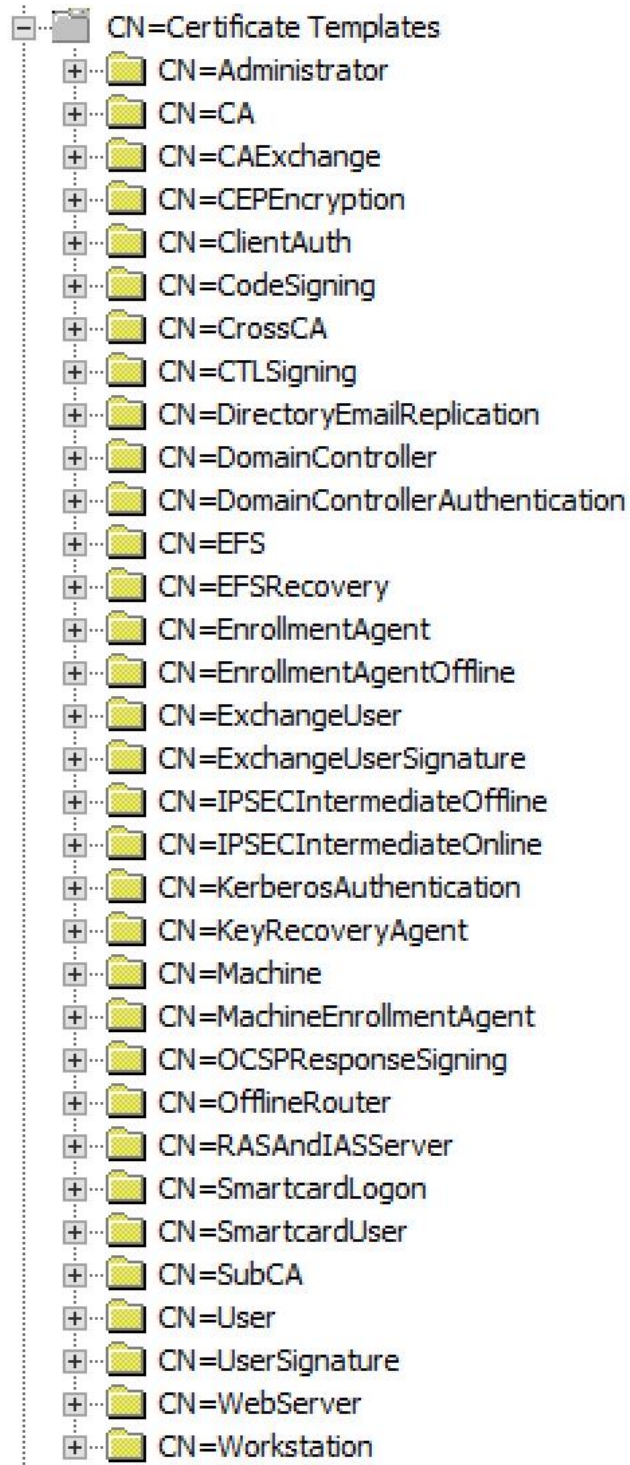
CA 证书类的 objectClass 属性为 certificationAuthority, cACertificate 属性为 CA 证书的二进制内容。如图所示, 是 CA 证书颁发机构 xie-CA-SERVER-CA 的属性:

Attribute	Syntax	Count	Value(s)
authorityRevocationList	OctetString	1	0
cACertificate	OctetString	1	48 130 3 101 48 130 2 77 160 3 2 1 2 2 16 17 253 128 1
certificateRevocationList	OctetString	1	0
cn	DirectoryString	1	xie-CA-SERVER-CA
distinguishedName	DN	1	CN=xie-CA-SERVER-CA,CN=Certification Authorities,CN
dSCorePropagationData	GeneralizedTime	1	1601/1/1 7:00:00
instanceType	Integer	1	4
name	DirectoryString	1	xie-CA-SERVER-CA
nTSecurityDescriptor	NTSecurityDescriptor	1	D:AI(A;OICI;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;D
objectCategory	DN	1	CN=Certification-Authority,CN=Schema,CN=Configurati
objectClass	OID	2	top;certificationAuthority
objectGUID	OctetString	1	{9F0D120E-C5A7-4955-8299-507DA6734E25}
showInAdvancedViewO...	Boolean	1	TRUE
usnChanged	Integer8	1	0x3213
usnCreated	Integer8	1	0x3213
whenChanged	GeneralizedTime	1	2022/6/22 22:37:54
whenCreated	GeneralizedTime	1	2022/6/22 22:37:54

Attribute Properties	
Attribute:	cACertificate
Object:	CN=xie-CA-SERVER-CA,CN=Certification Authorities,CN=Public Key Services,CN=Services,CN=Configuration,C
Syntax:	OctetString
Schema:	CN=CA-Certificate,CN=Schema,CN=Configuration,DC=xie,DC=com Go to
Values:	48 130 3 101 48 130 2 77 160 3 2 1 2 2 16 17 253 128 139 178 229 206 176 69 149 219 126 77 97 20 124 48 13 6 9 42 13 7 87 0 128 78 42 174 118 119 214 160 189 4 254 230 177 76 57 121 67 150 58 150 189 58 35 72 49 130 90 80 136 18 20 1 63 196 2 193 216 155 41 23 7 30 222 250 208 205 10 253 241 70 27 221 239 105 108 0 246 73 46 179 10 123 218 214 2

Certificate Templates

Certification Authorities 是证书模板类, 里面的实例都是证书模板, 如图所示:



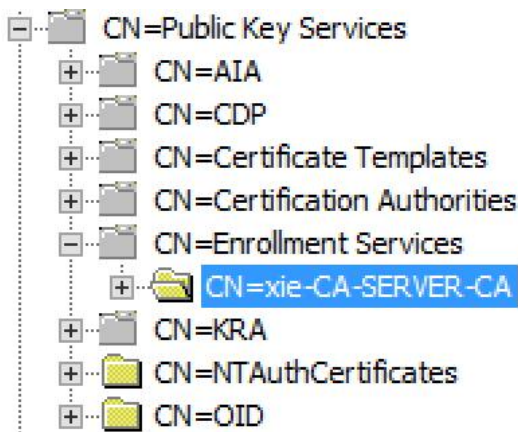
证书模板的 objectClass 属性为 pKICertificateTemplate，如图所示是客户端认证的证书模板的属性：

Path: CN=ClientAuth,CN=Certificate Templates,CN=Public Key Services,CN=Services,CN=Configuration,DC=xie,DC=com,10.211.55.4 [AD01.xie.com]

Attribute	Syntax	Count	Value(s)
displayName	DirectoryString	1	通过身份验证的会话
distinguishedName	DN	1	CN=ClientAuth,CN=Certificate Templates,CN=Public Key Services,CN=Services,CN=Configuration,DC=xie,DC=com
dSCorePropagationData	GeneralizedTime	1	1601/1/1 7:00:00
flags	Integer	1	66080
instanceType	Integer	1	4
msPKI-Cert-Template-OID	DirectoryString	1	1.3.6.1.4.1.311.21.8.5337220.12856170.14961318.2620336.1913209.31.1.4
msPKI-Certificate-Name...	Integer	1	-2113929216
msPKI-Enrollment-Flag	Integer	1	32
msPKI-Minimal-Key-Size	Integer	1	2048
msPKI-Private-Key-Flag	Integer	1	0
msPKI-RA-Signature	Integer	1	0
msPKI-Template-Minor...	Integer	1	1
msPKI-Template-Schem...	Integer	1	1
name	DirectoryString	1	ClientAuth
nTSecurityDescriptor	NTSecurityDescriptor	1	D:P(AI(OA);RPWPQR;0e10c968-78fb-11d2-90d4-00c04f79dc55;;DA)(OA);RPWPQR;0e10c968-78fb-11d2-90d4-00c04f79dc55;;DU)(OA);RPWPQR;0e10c968-78fb...
objectCategory	DN	1	CN=PKI-Certificate-Template,CN=Schema,CN=Configuration,DC=xie,DC=com
objectClass	OID	2	top,pKI-CertificateTemplate
objectGUID	OctetString	1	{AB2AE9AC-EEEF-4E85-8817-2A21A7983C57}
pKIExtensions	DirectoryString	1	2.5.29.15
pKIDefaultCSPs	DirectoryString	3	3,Microsoft Base DSS Cryptographic Provider;2,Microsoft Base Cryptographic Provider v1.0;1,Microsoft Enhanced Cryptographic Provider v1.0
pKIDefaultKeySpec	Integer	1	2
pKIExpirationPeriod	OctetString	1	0 64 57 135 46 225 254 255
pKIExtendedKeyUsage	DirectoryString	1	1.3.6.1.5.5.7.3.2
pKIKeyUsage	OctetString	1	128 0
pKIKeyUsageDepth	Integer	1	0
pKIOverlapPeriod	OctetString	1	0 128 166 10 255 222 255 255
revision	Integer	1	3
showInAdvancedViewO...	Boolean	1	TRUE
uSNChanged	Integer8	1	0x807A

Enrollment Services

Enrollment Services 是企业 CA 类，里面的实例都是企业 CA，如图所示：



企业 CA 的 objectClass 属性为 pKIErollmentService、dNSHostName 属性是企业 CA 的主机名、cACertificate 属性值包含 CA 证书的二进制内容、certificateTemplates 字段定义了启用的证书模板。证书模板是 CA 在创建证书时使用的设置“蓝图”，包括 EKU、注册权限、证书到期、颁发要求和加密设置等内容。如图所示是企业 CA xie-CA-Server-CA 的属性：

Attribute	Syntax	Count	Value(s)
 cACertificate	OctetString	1	48 130 3 101 48 130 2 77 160 3 2 1 2 2 16 17 253 128 139 178 229 206 176 69
 cACertificateDN	DirectoryString	1	CN=xie-CA-SERVER-CA, DC=xie, DC=com
 certificateTemplates	DirectoryString	12	ESC1;DirectoryEmailReplication;DomainControllerAuthentication;KerberosAuth
 cn	DirectoryString	1	xie-CA-SERVER-CA
 displayName	DirectoryString	1	xie-CA-SERVER-CA
 distinguishedName	DN	1	CN=xie-CA-SERVER-CA,CN=Enrollment Services,CN=Public Key Services,CN=S
 dNSHostName	DirectoryString	1	CA-Server.xie.com
 dSCorePropagationData	GeneralizedTime	5	2022/10/10 14:43:27;2022/10/10 14:43:17;2022/10/10 10:43:43;2022/10/10
 flags	Integer	1	10
 instanceType	Integer	1	4
 name	DirectoryString	1	xie-CA-SERVER-CA
 nTSecurityDescriptor	NTSecurityDescriptor	1	D:AI(OA;;CR;0e10c968-78fb-11d2-90d4-00c04f79dc55;;AU)(A;OICI;CCDCLCS
 objectCategory	DN	1	CN=PKI-Enrollment-Service,CN=Schema,CN=Configuration,DC=xie,DC=com
 objectClass	OID	2	top;pKIEnrollmentService
 objectGUID	OctetString	1	{C89F8C4F-CCDA-4335-8549-16E4DA0F0354}
 showInAdvancedViewO...	Boolean	1	TRUE
 uSNChanged	Integer8	1	0x6060
 uSNCreated	Integer8	1	0x321F
 whenChanged	GeneralizedTime	1	2022/10/10 14:43:27
 whenCreated	GeneralizedTime	1	2022/6/22 22:37:54

NTAuthCertificates

该容器定义了有资格颁发身份验证证书的 CA 证书。这个对象的 objectClass 属性值为 certificateAuthority，并且 cACertificate 属性定义了一个可信 CA 证书的二进制数组。加入 AD 的 Windows 机器将这些 CA 证书下发到每台机器上的中间证书颁发机构证书存储区。仅当 NTAuthCertificates 对象定义的 CA 签署了身份验证客户端的证书时，客户端应用程序才能使用证书向 AD 进行身份验证。

Attribute	Syntax	Count	Value(s)
 authorityRevocationList	OctetString	1	0
 cACertificate	OctetString	1	48 130 3 101 48 130 2 77 160 3 2 1 2 2 16 17 253 128 139 178 229 206 176 69 149 219 126 77 5
 certificateRevocationList	OctetString	1	0
 cn	DirectoryString	1	NTAuthCertificates
 distinguishedName	DN	1	CN=NTAuthCertificates,CN=Public Key Services,CN=Services,CN=Configuration,DC=xie,DC=com
 dSCorePropagationData	GeneralizedTime	1	1601/1/1 7:00:00
 instanceType	Integer	1	4
 name	DirectoryString	1	NTAuthCertificates
 nTSecurityDescriptor	NTSecurityDescriptor	1	D:AI(A;OICI;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;DA)(A;OICI;CCDCLCSWRPWPDTLOCRSDF
 objectCategory	DN	1	CN=Certification-Authority,CN=Schema,CN=Configuration,DC=xie,DC=com
 objectClass	OID	2	top;certificationAuthority
 objectGUID	OctetString	1	{C1B9BF05-4D99-43D1-B4A5-B3BAF0F20952}
 showInAdvancedViewO...	Boolean	1	TRUE
 uSNChanged	Integer8	1	0x3215
 uSNCreated	Integer8	1	0x3214
 whenChanged	GeneralizedTime	1	2022/6/22 22:37:54
 whenCreated	GeneralizedTime	1	2022/6/22 22:37:54

AIA (Authority Information Access)

该容器保存了中间 CA 证书的 AD 对象。中间 CA 是 PKI 树层次结构中根 CA 的“子代”，因此，此容器的存在是为了帮助验证证书链。与 Certification Authorities 容器一样，每个 CA 在 AIA 容器中表示为一个 AD 对象，其中 objectClass 属性设置为 CertificationAuthority，并且 cACertificate 属性包含 CA 证书的二进制内容。当有新的 CA 安装时，它的证书则会自动放到 AIA 容器中。这些 CAs 传播到每台机器上的中间证书颁发机构证书存储区。

Attribute	Syntax	Count	Value(s)
cn	DirectoryString	1	AIA
distinguishedName	DN	1	CN=AIA,CN=Public Key Services,CN=Services,CN=Configuration,DC=xie,DC=com
dSCorePropagationData	GeneralizedTime	1	1601/1/1 7:00:00
instanceType	Integer	1	4
name	DirectoryString	1	AIA
nTSecurityDescriptor	NTSecurityDescriptor	1	D:AI(A;;LCRPLOCR;;;AU)(A;;CCLCSWRPWPLOCRRCDWO;;;EA)(A;;CCDCLCSWRPWPDTLOCR;
objectCategory	DN	1	CN=Container,CN=Schema,CN=Configuration,DC=xie,DC=com
objectClass	OID	2	top;container
objectGUID	OctetString	1	{F5A1A9DB-A288-4B43-B4E2-D216EDAECB08}
showInAdvancedViewO...	Boolean	1	TRUE
uSNCreated	Integer8	1	0x1027
uSNCreated	Integer8	1	0x1027
whenChanged	GeneralizedTime	1	2022/5/12 23:27:47
whenCreated	GeneralizedTime	1	2022/5/12 23:27:47

NTAuthCertificates 对象

在使用证书进行 PKINIT Kerberos 认证期间，DC 可以检查身份验证证书是否链接到由 NTAuthCertificates (CN=NTAuthCertificates,CN=Public Key Services,CN=Services,CN=Configuration,DC=xie,DC=com)对象定义的 CA 证书，因此可以认为 NTAuthCertificates 对象中的 CA 证书必须依次链接到根 CA，NTAuthCertificates 对象是 Active Directory 中证书身份验证的信任根。

CN=NTAuthCertificates,CN=Public Key Services,CN=Services,CN=Configuration,DC=xie,DC=com,10.211.55.4 [DC01.xie.com]

Attribute	Syntax	Count	Value(s)
authorityRevocationList	OctetString	1	0
certificate	OctetString	1	48 130 3 101 48 130 2 77 160 3 2 1 2 2 16 17 253 128 139 178 229 206 176 69 149 219 126 77 97 20 124 48 13 6 9 42 13
certificateRevocationList	OctetString	1	0
cn	DirectoryString	1	NTAuthCertificates
distinguishedName	DN	1	CN=NTAuthCertificates,CN=Public Key Services,CN=Services,CN=Configuration,DC=xie,DC=com
dSCorePropagationData	GeneralizedTime	1	1601/1/1 7:00:00
instanceType	Integer	1	4
name	DirectoryString	1	NTAuthCertificates
nTSecurityDescriptor	NTSecurityDescriptor	1	D:AI(A;OICI;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;DA)(A;OICI;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;EA)(A;OICI;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;DA)
objectCategory	DN	1	CN=Certification-Authority,CN=Schema,CN=Configuration,DC=xie,DC=com
objectClass	OID	2	top;certificationAuthority
objectGUID	OctetString	1	{C1B9BF05-4D99-43D1-84A5-83BAF0F20952}
showInAdvancedViewOnly	Boolean	1	TRUE
uSNCreated	Integer8	1	0x3215
uSNChanged	Integer8	1	0x3214
whenChanged	GeneralizedTime	1	2022/6/22 22:37:54
whenCreated	GeneralizedTime	1	2022/6/22 22:37:54

不同后缀的证书文件

我们经常会看到不同后缀格式的证书，如下是不同后缀格式证书包含的内容：

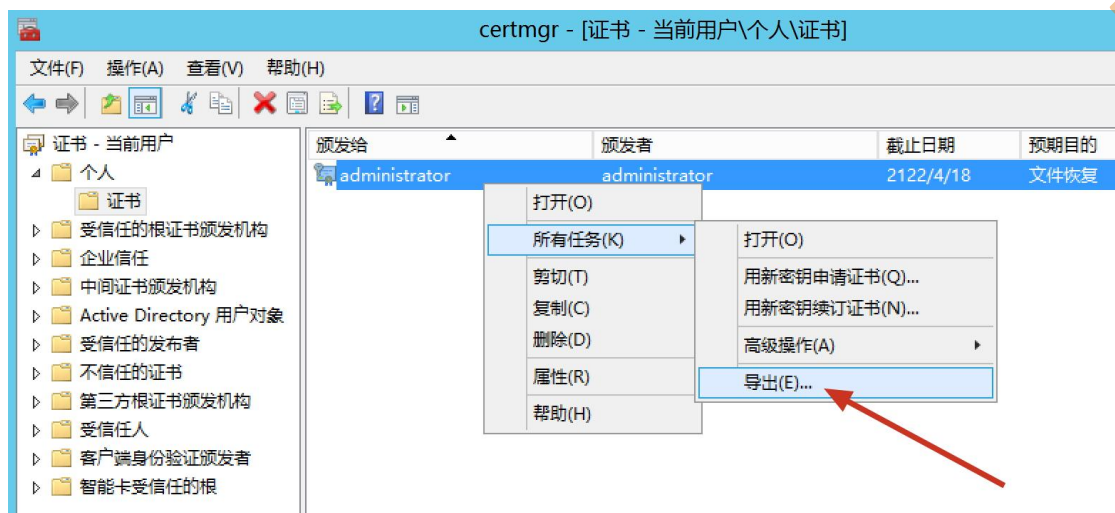
- .pfx、.p12、.pkcs12 后缀：包含证书和私钥，最有用。会有密码保护，导出该类型证书时会要求你输入一个密码。
- .pem 后缀：含有证书和私钥。
- .cer/.crt 后缀：包含证书。
- .key 后缀：只包含私钥。
- .csr 后缀：证书申请文件，既不包含公钥也不包含私钥。
- .jks/.keystore/.keys：Java 密钥库，可能包含 Java 应用程序使用的 certs + 私钥。

.pfx 格式的文件 ——> .pem 格式的文件。

.pem 格式的文件中 ——> .cer 证书文件 + .key 私钥

导出 pfx 证书文件

选中证书，导出



勾选将私钥和证书一起导出

  证书导出向导

导出私钥

你可以选择将私钥和证书一起导出。

私钥受密码保护。如果要私钥跟证书一起导出，你必须在后面一页上键入密码。

你想将私钥跟证书一起导出吗？

☒ 是，导出私钥(Y)

☐ 不，不要导出私钥(O)

下一步(N)

取消



这里需要输入一个密码，我们输入任意的 123456 即可。



最后我们生成包含私钥和证书的 administrator.pfx 文件

从.pfx 文件中提取出包含私钥和证书的.pem 文件

执行如下命令从 pfx 证书文件中导出 server.pem 文件，该 pem 文件包含私钥和证书。

```
openssl pkcs12 -in administrator.pfx -nodes -out administrator.pem
```

#此时会要求你输入密码，这个密码是将证书导出时的密码

→ Desktop openssl pkcs12 -in administrator.pfx -nodes -out administrator.pem

Enter Import Password:

MAC verified OK

证书导出时输入的密码

→ Desktop cat administrator.pem

Bag Attributes

localKeyID: 01 00 00 00

friendlyName: 2bad1cb0-61b1-4716-bc30-1112047d1022

Microsoft CSP Name: Microsoft Enhanced Cryptographic Provider v1.0

Key Attributes

X509v3 Key Usage: 10

-----BEGIN PRIVATE KEY-----

```

MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBAwggSjAgEAAoIBAQDFsee4Nlia6Ly0
Agjk9jck2xf2DmYWU54jt0PtU3Te8J0IH5Uir0sS/MpWR+R+I85BAmv34MoW4yU
RNMTwvMIC4B1ie8CsEmxnHhUBX2917d1rtuNeJi8ECN8x1IoClhPLxubeW/99hp
Oab5bj1UU4SWoez78brjpPxHKHjgzZu5B/7tkDGfCArYUERP3cFHGFZX1/qtwl
/OokluQMMyDATy7DP7LnDmxeHvrWWRaGi/kuC8iSNthIyvTDBA/gSfMEiwdOfmL
zjUkB1cWgB51TxczVZk3Ly46Nn2XXLj5deZFT89aNS9VtbsgzFa1/j9S240NumfZ
04RstifdAgMBAAECggEAFy4KLAT/pCA0VZFDAjelVRa0W0cqbKU+x1DLb/E6HZbI
TsbH0YLdaENHTc6Tn02t22FQEnVKX13GoK7mMeu6i7+vfviiek+ngJiliyXVAX1VP
/0Yym4afn/NjedK0HRpDawdVRbxwGx74GrRmEdp9aekGA5R2YT8hauYyJ1TFQZS0
C1gBEs0GKaBV3wX0TY4wZ6n+5dMrPVwN6YNXMc0vx1YLLWvkb1axgHo2LgsgMTHr
TV1VG5YC7+ymEoJ7bSCKJyVBNj6wMtWakN+XcI+1kB9gl4uZPPVuTEl3oYyDFgx
Nb0PatyEdSMwz0lkXM700908uPmVL SobJTRn3PtFsQKBgQDr2Qq0sLC7Dyk1qzAn
ARXCzb1HZB2oYXVhKxTDahYD6UJZraNo6kDZr2GsRTVtKovyyfwKsRXuTzFSXdY5G
mdyMywYdTqbeJQgqdQqi5mnk7Ryxbu2N7Xze93jPGSODZlvpIY2xU6Ev4Q3CPWk
RTt7pA0hy6oGt3odM2uA6wkpKQKBgQDWLk23BmaUUMWnq7fmjSs8TqKxCUZLVK+F
PSUk2Y601K8s4NGtP5Bo6u02SZE2TR00eV3P058I7WA2yfTtF51LHQP0Cseag1B4
oDsEjHb16ek0N0mH36PKC0cQL1onF7za2LIuqmRgPtZRU4gS2m1t3c9qGGcihGcT
lBb5EwyTjQKBgQDW3vq9/GxHQFzPJJPONVLZvfcaX8AVdBU43cMg9jnwfkfF+FU6
HcE3KAKdZnJp4WxQQx7wTK11ziCThmqFMBcbxSxg74V58bUivRcrW9w1rIAuQK/w
J3mBx0w0hwU11oYAD0iQzCBPeRy/9020x86VqrS8IY1aaTa7++jpnzjNwQKBgAma
fm3ja901bbMcT0WlKbswNb3AkcxDxPssil4E0o11BAX5JckgICFh6cD9t977QoRX
rigZHfvzS6wIC8sZ0/vh14yzcs1qaZmMajfA5Y4J4uLwP0dSbe0b5h1GVT2MpWoR
6p+GBqn8BfFr0JovZlhVZfP0laEI1egdZ43MGXL9AoGARKBSTKmBvLPh+Y00psbc
GGwj2VKhZSP+/3M8gNR0eJ1lCJLVHULoW9NCdDL9SIAbWfyzmEil5jY/qcYpA3ui
c1UNPpFZqRuTLuLp2BxScDyMhmnIHlyN3imv6cylEnz2BQL3zmheZmj8MxtY5L8
+XNERxE5G/e6RGMff9vxZAM=

```

-----END PRIVATE KEY-----

Bag Attributes

localKeyID: 01 00 00 00

subject=/CN=administrator/L=EFS/OU=EFS File Encryption Certificate

issuer=/CN=administrator/L=EFS/OU=EFS File Encryption Certificate

-----BEGIN CERTIFICATE-----

```

MIIDgDCCAmigAwIBAgIQNsFYa0TkK4dGCjZbYHRfBTANBgkqhkiG9w0BAQUFADBQ
MRYwFAYDVQQDEw1hZG1pbm1ldHJhdG9yMQwwCgYDVQQHEwNFRlMxKDAmBgNVBAS
T H0VGuYBGAWxlIEVUy3J5cHRpb24gQ2VydG1maWNhdGUwIBcNMjIwNTEyMTUzMTI1
WhgPMjE5MjA0MTgxNTMxMjVAMFAxZjAUBGNVBAWTDWfkbWLuXN0cmF0b3IxDDAK

```

如图可以看到，如下包含的就是私钥，可以将其保存为 private.key 文件。

```
-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBCcwggSjAgEAAoIBAQDFsee4Nlia6LyO
Agjk9jck2xf2DmYWU54jt0PtU3Te8J0IH5UirOsS/MMpWR+R+I85BAmv34MoW4yU
RNMTwwMIC4B1ie8CsEmxnHhUBX2917d1rtuNeJi8ECN8x1loClhPILxubeW/99hp
Oab5bj1UU4SWoez78brjpPxHKHjgzZu5B/7tkDGfCArYUERP3cFHGFZX1/qtwl
/OokluQMMMyDAAty7DP7LnDmxeHvrWWRaGi/kuC8iSNthlyvTDBA/gSfMEiwdOfml
zjUkB1cWgB51TxczVZk3Ly46Nn2XXLj5deZFT89aNS9VtbsgzFa1/j9S240NumfZ
04RstifdAgMBAAECggEAFy4KLAT/pCA0VZFDAjeIVRa0W0cqbkU+xIDLb/E6HZbl
TsbHOYLdaENHTc6Tn02t22FQEnVKX13GoK7mMeu6i7+vfiek+ngJiliyXVAXIVP
/0Yym4afn/NjedK0HRpDawdVRbxwgx74GrRmEdp9aekGA5R2YT8hauYyJ1TFQZSO
C1gBEsOGKaBV3wXOTY4wZ6n+5dMrPVwN6YNXMc0vx1YLLWvkb1axgHo2LgsqMT
Hr
TV1VG5YC7+ymeOj7bSCKJyVBNj6wvMtWakN+Xcl+1kB9gl4uZPPVuTEI3oYYdFgx
NbOPatyEdSMwz0lkXM7O0908uPmVISobJTRn3PtFsQKBgQDr2Qq0sLC7Dyk1qzAn
ARXCbz1HZB2oYXVhKxTDahYD6UZraNo6kDZr2GsRTVtKovyyfwKsRXuTzFSXdY5G
mdyMywYdTqbeJQgqdQqi5mnk7Ryxbu2N7Xze93jPGSODZlvpIY2xU6Ev4Q3CPWk
RTt7pA0hy6oGt3odM2uA6wkpkQKBgQDWlk23BMAUUMWnq7fmjSs8TqKxCUzLVK+
F
PSUk2Y6OIK8s4NGtP5Bo6uO2SZE2TR00eV3PO58I7WA2yfTtf51LHQP0CseaglB4
oDsEjHb16ek0NOMH36PKC0cQL1onF7za2LluqmRgPtZRU4gS2m1t3c9qGGcihGcT
IBb5EwyTjQKBgQDW3vq9/GxHQFzPJJPONVLZvfcaX8AVdBU43cMg9jnwftF+FU6
HcE3KAKdZnJp4WxQQx7wTK11zicThmqFMBCbxSxg74V58bUivRcrW9w1rlAuQK/w
J3mBx0w0hwUI1oYADOiqZcBPeRy/9020x86VqrS8IY1aaTa7++jpnzjNwQKBgAma
fm3ja9O1bbMcT0WIKbswNb3AkcxDPssil4E0ol1BAX5JckglCFh6cD9t977QoRX
rigZHfzS6wlC8sZO/vhl4yzcs1qaZmMajfA5Y4J4uLwP0dSbe0b5hIGVT2MpWoR
6p+GBqn8BfFr0JovZlhVZfP0laEI1egdZ43MGXL9AoGARKBSTKmBvLPh+YOOpsbc
GGwj2VKhZSP+/3M8gNROeJ1ICJLVHULoW9NCdDL9SlaBWFyzmEil5jY/qcYpA3ui
clUNPpFZqRuTLuLp2BxScDyMhmnIHlyN3imv6cylEnz2BQL3zmheZmj8MxtY5L8
+XNERxE5G/e6RGMff9vxZAM=
-----END PRIVATE KEY-----
```

如下包含的就是证书，可以将其保存为 administrator.cer 文件。

```
-----BEGIN CERTIFICATE-----
MIIDgDCCAmigAwIBAgIQNsFYa0TkK4dGCjZbYHRfBTANBgkqhkiG9w0BAQUFADBQ
MRYwFAYDVQQDEw1hZG1pbmlzdHJhdG9yMQwwCgYDVQQHEwNFRIMxKDAmBgN
VBAsT
H0VGUyBGaWxlIEVY3J5cHRpb24gQ2VydGlmZWVhdGUwBcNMjIwNTEyMTUzMTI1
WhgPMjE5MjA0MTgxNTMxMjVAMFAxZjAUBGNVBAWTDWFKbWluaXN0cmF0b3IxDD
AK
BgNVBAcTA0VGUzEoMCYGA1UECXMfRUZTIEZpbGUgRW5jcnlwdGlvbiBDZXJ0aWZ
p
Y2F0ZTCCASlwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAMWx57g2WJrovl
4C
COT2NyTbF/YOZhZTniO3Q+27dN7wnQgfISKs6xL8wylZH5H4jzkECa/fgyhbjJRE
0xPC8wgLgHWJ7wKwSbGceFQFfb3Xt3Wu2414mLwQl3zHUigKWE+UvG5t5b/32Gk
5
pvlUPVRThJah7PvxuuOk/EcoeODPO7kH/u2QMZ8ICtFi4RE9TdwUcYVlfX+q3CX8
6iSW5AwzIMC1PLsM/sucObF4e+tZZFoaL+S4LyJI22EjK9MMED+BJ8wSLB05+aXO
NSQHVxaAHnVPFzNVmTcvLjo2fZdcuPI15kVPz1o1L1W1uyDMVrX+P1LbjQ26Z9nT
hGy2J90CAwEAANUMFlwFgYDVR0IBA8wDQYLKwYBBAGCNwoDBAEwLQYDVR0R
BCYw
JKAiBgorBgEEAYI3FAIDoBQMEFkbWluaXN0cmF0b3JAWEIFADAJBgNVHRMEAjAA
MA0GCSqGSIb3DQEBBQUAA4IBAQBIZrrlsFFKrxpqgiyi8G0TbnRTMKIVxwM238x
709mMtkOgG9Xv0xIK5flkRD9PILZGEi1wEEr1zNK9exSXF9NcoeJ13GzEpm6Ledj
2PCS9cZIwCdZ4UdrHSOTB1PY9RRFb7Q6vQiUfQ6F+5EawHbSKhd3FzUkjHfQcApv
fIF0jne6Edb+zgmGP49BnnsW6cR/s13gvyPdkKkw5clAwPcNQ7sOmgUuF8TSs5+f
A8+Jeu+BEn31YS25YWKaLH/rAXIY/wYX6GXm96FAaUI6gmWxk0qBP6RFFvhHb/90
gb5vfdQIW9PO2M09fHfDFdmk3Jwuzms2JI6YIRwNxEvtQgQ
-----END CERTIFICATE-----
```

从.pem 文件中提取出.cer 证书文件

```
openssl x509 -in administrator.pem -out administrator.cer
```


→ Desktop openssl x509 -in administrator.pem -out administrator.cer
 → Desktop cat administrator.cer
 -----BEGIN CERTIFICATE-----
 MIIDgDCCAmigAwIBAgIQNsFYa0TkK4dGCjZbYHRfBTANBgkqhkiG9w0BAQUFADBQ
 MRYwFAYDVQQDEw1hZG1pbmVzdHJhdG9yMQwwCgYDVQQHEwNFRlMxKDAmBgNVBASt
 H0VGUyBGaWxlIEVuY3J5cHRpb24gQ2VydGlmYWVhdGUwIBcNMjIwNTEyMTUzMTI1
 WhgPMjEyMjA0MTgxNTMxMjVhMFAxZjAUBgNVBAMTDWFKbWluaXN0cmF0b3IxDDAK
 BgNVBACTA0VGUzEoMCYGA1UECxmFRUZYTIETZpbGUgRW5jcmlwdGlvbiBDZXJ0aWZp
 Y2F0ZTCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAMWx57g2WJrovI4C
 COT2NyTbF/Y0ZhZTni03Q+27dN7wnQgfLSKs6xL8wylZH5H4jzkECa/fgyhbJjRE
 0xPC8wglGHWJ7wKwSbGceFQFfb3Xt3Wu2414mLwQI3zHUiGKWE+UvG5t5b/32Gk5
 pvluPVRThJah7Pvxuu0k/EcoeODP07kH/u2QMZ8ICtFi4RE9TdwUcYVlfx+q3CX8
 6iSW5AwzIMC1PLsM/suc0bF4e+tZZFoaL+S4LyJI22EjK9MMED+BJ8wSLB05+aX0
 NSQHVxaAHnVPFzNVmTcvLjo2fZdcuPl15kVPz1o1L1W1uyDMVrX+P1LbjQ26Z9nT
 hGy2J90CAwEAANUMFIwFgYDVR0LBA8wDQYLKwYBBAGCNwoDBAEwLQYDVR0RBCYw
 JKAiBgorBgEEAYI3FAIDoBQMEFkbWluaXN0cmF0b3JAWElFADAJBgNVHRMEAjAA
 MA0GCSqGSIb3DQEBBQUAA4IBAQB17ZrrlsFFKrxpqi8G0TbnRTMKIVxwM238x
 709mMtk0gG9Xv0xLK5fLkRD9PLZGEi1wEEr1zNK9exSXF9NcoeJ13GzEpm6Ledj
 2PCS9cZIwCdZ4UdrHS0TB1PY9RRFb7Q6vQiUfQ6F+5EawHbSKhd3FzUkjhfQCApv
 fIF0jne6Edb+zgmGP49BnnsW6cR/s13gvyPdkKkw5cIAwPcNq7s0mgUuF8TSs5+f
 A8+Jeu+BEEn31YS25YWKaLH/rAXlY/wYX6GXm96FAaUI6gmWxk0qBP6RFFvhHb/90
 gb5vfdQIw9P02M09fHfDFdmk3Jwuzms2JI6YlRwNxEavtQgQ
 -----END CERTIFICATE-----

定位证书服务器

域内

在域内的话，可以执行如下命令定位证书服务器。

certutil.exe

#弹框

certutil -config - -ping

C:\Users\hack.XIE\Desktop>certutil.exe

Entry 0:

Name:	"xie-CA-SERVER-CA"
Organizational Unit:	""
Organization:	""
Locality:	""
State:	""
Country/region:	""
Config:	"CA-Server.xie.com\xie-CA-SERVER-CA"
Exchange Certificate:	""
Signature Certificate:	""
Description:	""
Server:	"CA-Server.xie.com"
Authority:	"xie-CA-SERVER-CA"
Sanitized Name:	"xie-CA-SERVER-CA"
Short Name:	"xie-CA-SERVER-CA"
Sanitized Short Name:	"xie-CA-SERVER-CA"
Flags:	"1"
Web Enrollment Servers:	""

CertUtil: -dump command completed successfully.

C:\Users\hack.XIE\Desktop>certutil -config - -ping

选择证书颁发机构

?

×

选择你要使用的证书颁发机构(CA)(S)。

CA

计算机



xie-CA-SERVER-CA

CA-Server.xie.com

如果是有多多个 CA，比如根 CA 下有从属 CA


```
C:\Users\test\Desktop>certutil.exe
```

```
Entry 0:
```

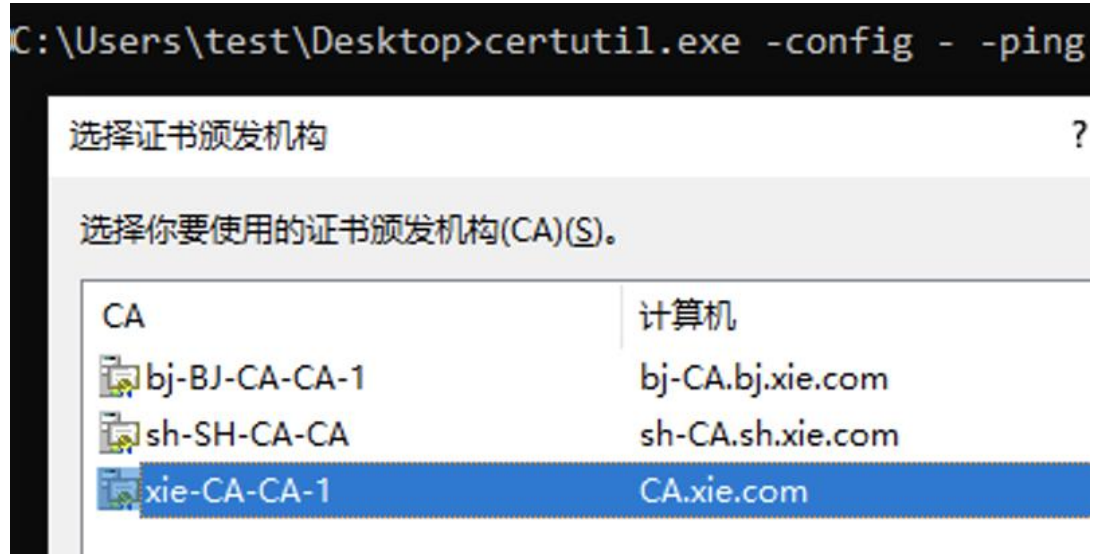
```
Name: "xie-CA-CA-1"
Organizational Unit: ""
Organization: ""
Locality: ""
State: ""
Country/region: ""
Config: "CA.xie.com\xie-CA-CA-1"
Exchange Certificate: ""
Signature Certificate: ""
Description: ""
Server: "CA.xie.com"
Authority: "xie-CA-CA-1"
Sanitized Name: "xie-CA-CA-1"
Short Name: "xie-CA-CA-1"
Sanitized Short Name: "xie-CA-CA-1"
Flags: "1"
Web Enrollment Servers: ""
```

```
Entry 1:
```

```
Name: "sh-SH-CA-CA"
Organizational Unit: ""
Organization: ""
Locality: ""
State: ""
Country/region: ""
Config: "sh-CA.sh.xie.com\sh-SH-CA-CA"
Exchange Certificate: ""
Signature Certificate: ""
Description: ""
Server: "sh-CA.sh.xie.com"
Authority: "sh-SH-CA-CA"
Sanitized Name: "sh-SH-CA-CA"
Short Name: "sh-SH-CA-CA"
Sanitized Short Name: "sh-SH-CA-CA"
Flags: "1"
Web Enrollment Servers: ""
```

```
Entry 2:
```

```
Name: "bj-BJ-CA-CA-1"
Organizational Unit: ""
Organization: ""
Locality: ""
State: ""
Country/region: ""
```



可以通过 nslookup 命令指定 DNS 服务器为域控进行解析 CA 服务器的 IP。

```
nslookup ca.xie.com 172.16.200.80
```

#如果是通过代理，则可以通过 -vc 参数指定使用 tcp 协议进行查询

```
nslookup -vc ca.xie.com 172.16.200.80
```

```
(root@Kali)-[~]  
# nslookup ca.xie.com 172.16.200.80  
Server:      172.16.200.80  
Address:     172.16.200.80#53
```

```
Name:  ca.xie.com  
Address: 172.16.200.82
```

```
(root@Kali)-[~]  
# nslookup sh-CA.sh.xie.com 172.16.200.80  
Server:      172.16.200.80  
Address:     172.16.200.80#53
```

```
Non-authoritative answer:  
Name:  sh-CA.sh.xie.com  
Address: 172.16.200.112
```

```
(root@Kali)-[~]  
# nslookup bj-CA.bj.xie.com 172.16.200.80  
Server:      172.16.200.80  
Address:     172.16.200.80#53
```

```
Non-authoritative answer:  
Name:  bj-CA.bj.xie.com  
Address: 172.16.200.92
```

```
→ ~ nslookup -vc ca.xie.com 172.16.200.80
Server:          172.16.200.80
Address:         172.16.200.80#53

Name:   ca.xie.com
Address: 172.16.200.82
```

域外

在域外的话，可以利用 certipy 工具执行如下命令定位证书服务器。

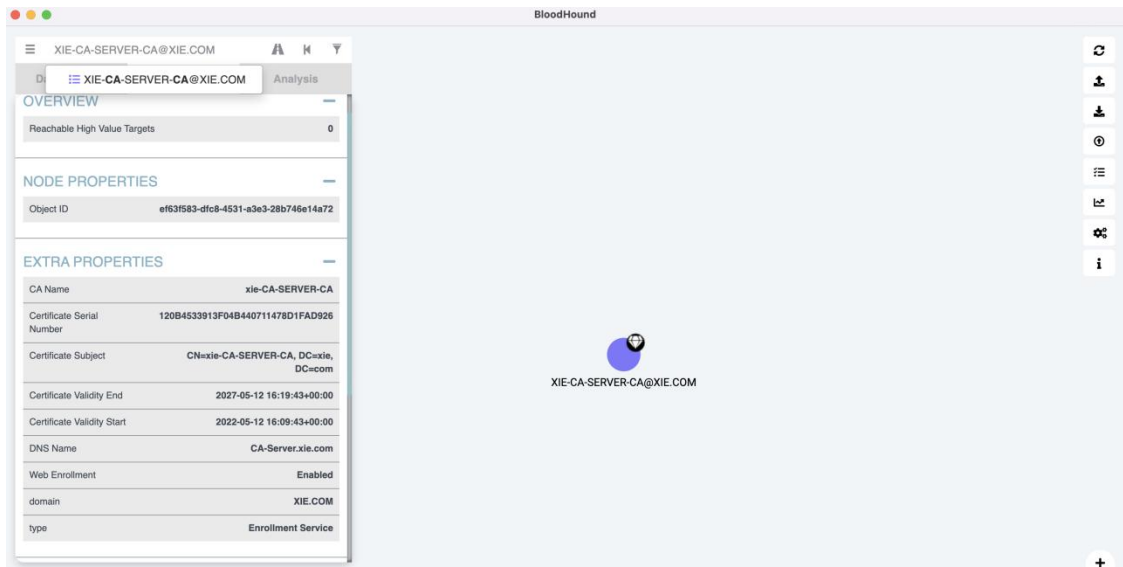
```
certipy find -u hack@xie.com -p P@ss1234 -dc-ip 10.211.55.4 -dc-only
```

执行完成后会在当前目录生成对应的 txt、json 和 zip 文件。查看 txt 和 json 文件，找到最主要的两个值：CA Name 和 DNS Name，这两个值我们后续需要用到。从图中可以看到 CA Name 和 DNS Name 分别为 xie-CA-SERVER-CA 和 CA-Server.xie.com。

```
→ Desktop certipy find -u hack@xie.com -p P@ss1234 -dc-ip 10.211.55.4 -dc-only
Certipy v4.0.0 - by Oliver Lyak (ly4k)

[*] Finding certificate templates
[*] Found 38 certificate templates
[*] Finding certificate authorities
[*] Found 1 certificate authority
[*] Found 15 enabled certificate templates
[*] Saved BloodHound data to '20221014221950_Certipy.zip'. Drag and drop the file into the BloodHound GUI from @ly4k
[*] Saved text output to '20221014221950_Certipy.txt'
[*] Saved JSON output to '20221014221950_Certipy.json'
→ Desktop cat 20221014221950_Certipy.json
{
  "Certificate Authorities": {
    "0": {
      "CA Name": "xie-CA-SERVER-CA",
      "DNS Name": "CA-Server.xie.com",
      "Certificate Subject": "CN=xie-CA-SERVER-CA, DC=xie, DC=com",
      "Certificate Serial Number": "11FD808BB2E5CEB04595DB7E4D61147C",
      "Certificate Validity Start": "2022-06-22 14:27:48+00:00",
      "Certificate Validity End": "2027-06-22 14:37:48+00:00",
      "Web Enrollment": "Unknown",
      "User Specified SAN": "Unknown",
      "Request Disposition": "Unknown"
    }
  },
  "Certificate Templates": {
    "0": {
      "Template Name": "ESC4",
      "Display Name": "ESC4",
      "Certificate Authorities": [
        "xie-CA-SERVER-CA"
      ]
    }
  }
}
```

也可以将.zip 文件导入 BloodHound 中进行分析。



如果想更深更细致的了解 AD 攻防知识，可以参加《域渗透攻防》培训课程：

https://mp.weixin.qq.com/s/Sjc_yTzB5CSYVk6bhnSsiQ

非常感谢您读到现在，由于作者的水平有限，编写时间仓促，文章中难免会出现一些错误或者描述不准确的地方，恳请各位师傅们批评指正。如果你想一起学习 AD 域安全攻防的话，可以加入下面的知识星球一起学习交流。



谢公子

我加入了这个星球，邀你一起学习



域渗透攻防指南

星主：谢公子

60+
成员数量

30+
内容数量

306
运营天数

书籍  《域渗透攻防指南》官方知识星球。
星球讨论与微软AD域相关攻防技术，大家互
相交流互相学习，共同进步成长。

现价：¥199

微信扫码加入星球



 知识星球

 谢公子学安全